

A Review of Some Classic Solvers

Several different notions and nomenclatures are used to describe linear solvers. This section connects Algorithm 17.2, in its general form or under certain restrictions, to some popular classic concepts. This will allow us, in Corollary 18.5, to identify a requirement that probabilistic solvers have to fulfil to be equivalent to CG.

► 18.1 Projection Methods

Independent of the concrete inference rule in line 12, Algorithm 17.2 selects the estimate x_i as $x_i \in x_0 + \mathbb{K}_i$ with $\mathbb{K}_i = \text{span}\{d_1, d_2, \dots, d_i\}$. That is, the estimate lies in an expanding sequence of sub-spaces spanned by the search-directions. Algorithms that select the estimate as $x_i \in x_0 + \mathbb{K}_i$, for some $\mathbb{K}_i$, are known as *projection methods*.¹ Because α_i is chosen to minimise the objective f along the search direction, the resulting gradient r_{i+1} is orthogonal to d_i (but not necessarily to all d_j for $j \leq i$). More generally, there is a space $\mathbb{L}_i$ orthogonal to the gradient,

$$\nabla f(x_i) = r_i = Ax_i - b \perp \mathbb{L}_i. \quad (18.1)$$

Methods for which $\mathbb{K} = \mathbb{L}$ are called *orthogonal projection methods*, and Eq. (18.1) is then known as the *Galerkin condition*.²

► 18.2 Conjugate Directions

A further restriction of orthogonal direction methods is formed by *conjugate directions* methods.

Definition 18.1 (Conjugate directions). *Given a symmetric matrix $A \in \mathbb{R}^{N \times N}$, two vectors $v, w \in \mathbb{R}^N$ are called A -conjugate to each other if $v^\top Aw = 0$. An iterative linear solver for $Ax = b$ with*

¹ Meister (2011), Def. 4.58

² Conversely, if $\mathbb{K} \neq \mathbb{L}$, the term *oblique-projection method* is occasionally used; Eq. (18.1) is then denoted as the *Petrov–Galerkin condition*. Galerkin’s name is attached to these methods because they relate to Galerkin methods, a family of solvers for partial differential equations that solve a high- or infinite-dimensional problem by *projecting* to a lower-dimensional space.

symmetric A is called a conjugate directions method if it chooses projections d_i that are pairwise A -conjugate to each other.

Conjugate directions methods converge to the correct solution x in at most N steps,³ a property called *linear consistency*. They also ensure that the gradient r_k after k steps is orthogonal to the preceding search directions,⁴ i.e. $r_k^\top p_i = 0$ for $i < k$. So they are orthogonal projection methods.

³ Nocedal and Wright (1999), Thm. 5.1

⁴ Nocedal and Wright (1999), Thm. 5.2

The following result connects our skeletal Algorithm 17.2 to conjugate direction methods. For the sake of readability, most technical derivations have been moved to the end of the chapter.

Theorem 18.2 (proof on p. 183). *If A is symmetric, and the inference rule in line 12 of Algorithm 17.2 produces a symmetric estimator H_i , then Algorithm 17.2 is a conjugate directions method.*

► 18.3 Krylov Subspace Methods

Another important sub-class of projection methods is formed by choosing $\mathbb{K}_i$ as the *Krylov sequence*

$$\mathbb{K}_i = \mathcal{K}^i(r_0, A) = \text{span}\{r_0, Ar_0, A^2r_0, \dots, A^{i-1}r_0\}. \quad (18.2)$$

Such algorithms are called *Krylov subspace methods*. They include the aforementioned seminal CG and GMRES. Krylov subspace methods have been studied in great detail since the 1980s. In addition to the textbooks already mentioned above, further information can also be found in the books by Demmel (1997), Greenbaum (1997), and Saad (2003). Due to the generic form of line 12, Algorithm 17.2 is not a Krylov subspace method in general, but we will see below what we need to do to make it one.

► 18.4 Conjugate Gradients

One characterisation of CG is that it is the conjugate direction method that is also a Krylov subspace method. Section 19.9 will identify a class of probabilistic models under which $\mathbb{K}_i$ is the Krylov sequence. Under this choice, Algorithm 17.2 will be shown to be equivalent to the method of conjugate gradients, based on the following technical result.

As in the preceding section, assume that the estimator H_i constructed in line 12 of Algorithm 17.2 is symmetric. Further, now assume that H_i is selected⁵ such that, for all $i \geq 1$,

⁵ Eq. (18.3) is obviously redundant, because the spans of S_i , Y_i , and r_i may overlap. This somewhat awkward form is deliberate, as it will later simplify the construction of concrete probability measures satisfying this restriction. Lemma 18.3 clears out the notational undergrowth, revealing that the assumption can be written much more succinctly.

$$d_i = -H_{i-1}r_{i-1} \in \text{span}\{s_1, \dots, s_{i-1}, y_1, \dots, y_{i-1}, r_{i-1}\}. \quad (18.3)$$

The following lemma establishes the connection to Krylov subspace methods.

Lemma 18.3 (proof on p. 184). *Assumption (18.3) is equivalent to the statement that Algorithm 17.2 is a Krylov subspace method, i.e. it is equivalent to the statement*

$$s_i \in \text{span}\{r_0, Ar_0, A^2r_0, \dots, A^{i-1}r_0\}. \quad (18.4)$$

We will now show the following Theorem.

Theorem 18.4 (proof on p. 184). *If A is SPD, H_i is symmetric for all $i \geq 0$, Assumption (18.3) holds, and Algorithm 17.2 does not terminate before step $k < N$, then*

$$r_i \perp r_j \quad \forall 0 \leq i \neq j \leq k,$$

and there exist $\gamma_i \in \mathbb{R}_{\setminus 0}$ for all $i < k$ so that line 12 in Algorithm 17.2 can be written as

$$d_i = -H_{i-1}r_{i-1} = \gamma_i \left(-r_{i-1} + \frac{\beta_i}{\gamma_{i-1}} d_{i-1} \right), \quad (18.5)$$

with

$$\beta_i := \frac{r_{i-1}^\top r_{i-1}}{r_{i-2}^\top r_{i-2}}.$$

Comparing Algorithm 17.2 to CG (Algorithm 16.1), we note that they are identical up to re-scaling by γ_i :

$$d_i^{\text{CG}} = \gamma_i d_i^{\text{Probabilistic}}.$$

Since the scaling $\gamma_i \in \mathbb{R}$ is cancelled by the step size α_i in line 7, the two algorithms produce the same sequence of estimates $\{x_i\}$, and we have the following result:

Corollary 18.5. *Under the assumptions of Theorem 18.4 on the estimators H_i , Algorithm 17.2 is equivalent to the method of conjugate gradients, in the sense that it produces the exact same sequence $\{x_0, x_1, \dots, x_N\}$ of estimates as CG initialised at $x_0 = H_0 b$.*

Figure 18.1 depicts how the gradients are sampled by CG – or equivalently, if the Assumptions of Theorem 18.4 are satisfied, by Algorithm 17.2.

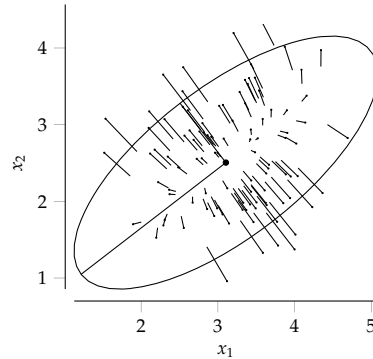


Figure 18.1: Analogous plot to Figure 17.1. The gradients at points sampled independent of the problem's structure ("needles" of point and gradient as black line, drawn from spherical Gaussian distribution around the extremum) are likely to be dominated by the eigenvectors of the largest eigenvalues. Thus, by following the gradient of the problem, one can efficiently compute a low-rank approximation of A that captures most of the dominant structure. This intuition is at the heart of the *Lanczos process* that provides the structure of conjugate gradients.

► 18.5 Preconditioning

Krylov subspaces (Eq. (18.2)) have the following invariance properties⁶ for scalars $\sigma, \tau \in \mathbb{R}_{\setminus 0}$ and orthonormal matrices U

⁶ Parlett (1980), §12.2.2

(i.e. $U^\top = U^{-1}$):

$$\begin{aligned} \text{scaling:} & \quad \mathcal{K}^m(\sigma r, \tau A) = \mathcal{K}^m(r, A), \\ \text{translation:} & \quad \mathcal{K}^m(r, A + \sigma I) = \mathcal{K}^m(r, A), \\ \text{change of basis:} & \quad \mathcal{K}^m(Ur, UAU^\top) = U\mathcal{K}^m(r, A). \end{aligned} \quad (18.6)$$

A generalisation of the last Equation (18.6) is the notion of a *preconditioned* iterative solver. Take a non-singular matrix C , and consider the transformed – the *pre-conditioned* – problem

$$\begin{aligned} \tilde{A}\tilde{x} &= \tilde{b} \quad \text{with} \\ \tilde{A} &:= C^{-\top}AC^{-1}, \quad \tilde{x} := Cx, \quad \tilde{b} = C^{-\top}b. \end{aligned} \quad (18.7)$$

If C is chosen smartly, the conjugate gradient method can converge significantly faster on the transformed system (18.7) than on the original one. As shown in Algorithm 18.1, preconditioning can be almost totally externalised from the basic solver, by adding only two additional lines, both solving a linear problem of the form $Ky = r_{k+1}$, with the matrix $K = CC^\top$. The ideal but impractical choice is $K = A$, which directly yields the exact solution in one step. Of course, having to compute the inverse C^{-1} of what would then be the matrix square root of A begs the question of running the solver in the first place. Good choices of pre-conditioners C are thus some form of incomplete or approximate decompositions of A that can be efficiently inverted.⁷ contains a general introduction to the topic and some specific examples.

⁷ For a general introduction to this topic and some specific examples, see Golub and Van Loan (1996), §11.5.

Let us return to the context of probabilistic solvers, and speak simplistically for the moment. Imagine that we find a probability measure $p_{\text{CG}}(A)$ over A consistent with the conjugate gradient method, in the sense of Theorem 18.4. Then consider a variation $p_{\text{PCG}}(A)$ of it that maps d_i to the transformed space

$$d_i \in \text{span}\{s_1, \dots, s_{i-1}, y_1, \dots, y_{i-1}, K^{-1}r_{i-1}\}. \quad (18.8)$$

Then, $p_{\text{PCG}}(A)$ is consistent with a preconditioned conjugate gradient method using the pre-conditioner $K = CC^\top$.

Since the process of preconditioning involves using an *a priori* guess K^{-1} for the matrix inverse H to simplify the work of the solver, it will be no surprise for us to find later (Corollary 19.17) that this modification in Eq. (18.8) is associated with a change to the prior of a probabilistic solver. And, with the derivations still to follow, we can already anticipate that the ideal choice $K = A$ will be associated with a particularly “natural” prior.

```

1 procedure PCG( $A(\cdot)$ ,  $b$ ,  $x_0$ )
2    $r_0 = Ax_0 - b$                                 // initial gradient
3    $g_0 = K^{-1}r_0$                                 // corrected gradient
4    $d_0 = 0, \beta_0 = 0$                             // for notational clarity
5   for  $i = 1, \dots, N$  do
6      $d_i = -g_{i-1} + \beta_{i-1}p_{i-1}$                 // compute direction
7      $z_i = Ad_i$                                 // compute
8      $\alpha_i = -d_i^\top r_{i-1} / d_i^\top z_i$           // optimal step-size
9      $s_i = \alpha_i d_i$                             // re-scale step
10     $y_i = \alpha_i z_i$                             // re-scale observation
11     $x_i = x_{i-1} + s_i$                             // update estimate for  $x$ 
12     $r_i = r_{i-1} + y_i$                             // new gradient at  $x_i$ 
13     $g_i = K^{-1}r_i$                                 // corrected gradient
14     $\beta_i = r_i^\top s_i / r_{i-1}^\top r_{i-1}$         // compute conjugate correction
15  end for
16 end procedure

```

Algorithm 18.1: Preconditioned conjugate gradients. Adapted from Nocedal and Wright (1999). The only difference to Algorithm 16.1 is the construction of the corrected gradient in lines 3 and 13.

► 18.6 Summary: Connecting Algorithm 17.2 to Classic Solvers

This section cast several families of classic linear solvers in the notation of Algorithm 17.2. Doing so identified certain properties of the prior measure $p(A)$ that make Algorithm 17.2 equivalent to these classic methods: any probabilistic estimator for the matrix inverse H which is merely consistent with observations can give rise to a projection method. If the estimate is also symmetric, Algorithm 17.2 becomes a conjugate directions method. Finally, we identified certain technical properties that make the algorithm equivalent to the method of conjugate gradients, and its preconditioned generalisation.

None of these restrictions directly imply a concrete form for $p(A)$; in particular, not a Gaussian one. So the situation in linear algebra differs from that in integration, where each quadrature rule was directly associated, up to a single scalar degree of freedom, with a particular Gaussian process prior on the integrand. The underlying abstract reason is that iterative linear solvers do a comparably simple job: an $N \times N$ invertible matrix has a *finite* number N^2 of degrees of freedom. At each iteration, the solver identifies N of them perfectly, while learning nothing about the other $N(N-1)$. Because of this, the *estimation* aspect of the computation is less pronounced than in, say, integration, where a finite number of evaluations must be used to estimate an infinite sum. Linear algebra, to a significant degree, amounts to sophisticated book-keeping rather than inferring an intractable

object.

This is not to say there is no use for uncertainty in linear solvers. It just so happens that classic solvers address a corner case, one less demanding of uncertainty. It is nevertheless useful to understand the connection to probabilistic inference in this domain, because uncertainty is more prominently important if:

- ⊖ the object of interest is an infinite-dimensional operator, rather than a finite matrix (this situation arises in the solution of partial differential equations); or
- ⊖ the question at hand involves the matrix inverse itself more than the estimate of the solution. For example if we want to compute Laplace approximations to large-scale models like deep neural networks,⁸ which involve the *inverse* Hessian of the regularised empirical loss. Such matrices are routinely much too large to be directly inverted. But they can also have a limited number of prominent eigenvalues. If we use an iterative solver in such situations, we will be forced to stop it very early compared to the size of the matrix, and then look for a good uncertainty estimate over the “unexplored” remainder; or
- ⊖ the projection observations are corrupted by noise. This situation arises regularly in machine learning applications when data is sub-sampled to lower computational cost. Sometimes such situations can be re-phrased as projections of a Schur complement, which is effectively a reduction to the previous points.

⁸ MacKay (1992); Daxberger et al. (2021).

A lack of restrictions is simultaneously a blessing and a curse for our search of a probabilistic interpretation of linear algebra. We may worry, if many quite different probabilistic priors can simultaneously be associated with, say, the conjugate gradient strategy, which is the *right* one? The answer we will find is: it depends. On the one hand, considerable additional prior assumptions will be required to arrive at a tractable uncertainty estimate. On the other, it will become clear that different kinds of priors may be useful to address different kinds of tasks.